

Variable/Keywords

Damit ein statisches YAML-Template zu einem ausführbaren Testfall wird, brauchen wir Variablen/Buffers

E-Variable

Um mit Agate Test Studio zu arbeiten, musst du dem System mitteilen, in welcher Umgebung du dich befindest. Das Setup ist in zwei Dateien unterteilt, die sich im `env/`-Ordner befinden:

1. Die `env/env.conf` (Die Welt deiner Instanzen)

Hier definierst du die "technischen Eckdaten" deiner Zielumgebungen. Jede Instanz beginnt mit einem eindeutigen Namen (z. B. `DEMOS` oder `ECS_SYST_AUT1`).

- **Wie füge ich eine neue Umgebung hinzu?** Kopiere einfach einen bestehenden Block und passe den Namen vor dem Punkt an.
- **Was ist wichtig?** Jede Zeile folgt dem Muster: `InstanzName.Parameter=Wert`. Hier legst du globale Werte fest, die für die gesamte Instanz gelten (z. B. IP-Adressen für Services oder Pfade zu Log-Check-Tools).

YAML-Beispiel:

```
1 | - type: CMD
2 |   op: EXEC
3 |   command: 'echo {E[env.database.user]}'
4 |   response: buff_out
```

Wie funktioniert es?

Die Eingabe der Testumgebung beim Start des Testfalls ist zwingend erforderlich, z. B.:

```
1 | startTests.bat Milenko DEMOS cmd cmd_engine_demo
```

Der Platzhalter `{E[env.database.user]}` bedeutet: Suche in der `env.conf` nach `DEMOS.database.user`. Wenn dort `DEMOS.database.user=D_DB` steht,

```
1 | DEMOS.database.user=D_DB
```

wird der Befehl zu: `echo D_DB`:

```
1 | >>> DSL      - type: CMD
2 | >>> DSL      op: EXEC
3 | >>> DSL      command: 'echo {E[env.database.user]}'
4 | >>> DSL      response: buff_out
5 | >>> CMD      : echo D_DB
6 | <<< OUT      : D_DB
7 | >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 56 ms
```

2. Die `env/users.conf` (Deine individuelle Arbeitsweise)

Während die `env.conf` das "Wo" definiert, legt die `users.conf` fest, "wer" mit welchen Daten arbeitet. Hier kannst du spezifische Variablen pro Benutzer innerhalb einer Instanz hinterlegen.

- **Wann brauche ich das?** Wenn ein Testfall spezifische Daten eines Benutzers benötigt (z.B. eine Seriennummer einer Karte oder eine spezifische IP-Adresse), wird dies hier definiert.
- **Wie sieht das aus?** `InstanzName.BenutzerName.VariablenName=Wert`

YAML Beispiel

```
1      # 2
2      - type: CMD
3        op: EXEC
4        command: 'echo {E[users.staat]}'
5        response: buff_out
```

Wie funktioniert es?

Beim Aufruf

```
1 startTests.bat Milenko DEMOS cmd cmd_engine_demo
```

sucht das System nach in `env/users.config` nach

```
1 DEMOS.Milenko.staat=AUT
```

Wenn dort `DEMOS.Milenko.staat=AUT` definiert ist, wird der Befehl zu: `echo AUT`:

```
1 >>> DSL      # 2
2 >>> DSL      - type: CMD
3 >>> DSL      op: EXEC
4 >>> DSL      command: 'echo {E[users.staat]}'
5 >>> DSL      response: buff_out
6 >>> CMD      : echo AUT
7 <<< OUT      : AUT
8 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 32 ms
```

Hinweis: Wird eine Variable nicht in den Konfigurationsdateien gefunden, meldet das System keinen expliziten Fehler. Die Variable bleibt im YAML-Feld als "unresolved" stehen. Der Benutzer muss dies bei der Durchführung anhand des fehlerhaften Ergebnisses selbst bemerken.

B-Variablen – Die lokale Konfiguration

Während die `E`-Variablen die globale Umgebung und den Benutzer betreffen, dienen die `B`-Variablen dazu, **lokale Parameter direkt innerhalb eines Testfalls** zu definieren. Sie sind das Werkzeug, um ein Template für einen spezifischen Testlauf zu konfigurieren.

Was sind B-Variablen?

B steht für "Base". Sie werden direkt in der YAML-Struktur des Testfalls definiert. Sie sind ideal, um Werte zu kapseln, die nur für diesen einen, spezifischen Testfall relevant sind, und verhindern, dass du jedes Mal die zentralen Konfigurationsdateien (`env.conf` oder `users.conf`) anpassen musst.

Struktur und Anwendung

Du definierst sie im `variables` -Block deines Test-YAMLs.

YAML-Beispiel:

```
1 | YAML
  |
  | # Hello World Demo
  | - id: TC Hello World
  |   description: Hello World
  |   stage: "*"
  |   priority: HIGH
  |   variables:
  |     retryCount: '3'
  |   steps:
  |     # 1
  |     - type: CMD
  |       op: EXEC
  |       command: 'echo {B[retryCount]}'
  |       response: buff_out
```

Ergebnis:

```
1 =====
2 TEST CASE: TC Hello World
3 DESC      : Hello World
4 STAGE     : * | PRI0: HIGH
5 -----
6 VARIABLES:
7   retryCount      = 3
8 -----
9
10 >>> DSL      # 1
11 >>> DSL      - type: CMD
12 >>> DSL      op: EXEC
13 >>> DSL      command: 'echo {B[retryCount]}'
14 >>> DSL      response: buff_out
15 >>> CMD      : echo 3
16 <<< OUT      : 3
17 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 31 ms
18 -----
19 TEST RESULT: TC Hello World 2 [PASSED]
20 Time: 0,03s | Steps: 2
21 -----
```

Wie funktioniert es? Wenn das Framework diesen Test lädt, erkennt der

`YamlPlaceholderResolver` den Platzhalter `{B[retryCount]}`. Er schaut in den lokalen

`variables` -Block des aktuellen Testfalls, findet den Wert `"3"` und ersetzt ihn direkt in der Ausführung.

Warum solltest du B-Variablen nutzen?

1. **Modularität:** Dein Test bleibt unabhängig von globalen Einstellungen. Du kannst das `timeout` für einen einzelnen, langsameren Testfall erhöhen, ohne andere Tests zu beeinflussen.
2. **Übersichtlichkeit:** Jeder Testfall trägt seine Konfiguration "im Rucksack" mit sich. Wenn du den Test an einen Kollegen schickst, hat er sofort alle notwendigen Parameter zur Hand.
3. **Wartbarkeit:** Änderungen an einem Testfall betreffen nur diesen einen Test, was das Risiko für Seiteneffekte in deiner Testsuite minimiert.

Hinweis: Im Gegensatz zu den `E` -Variablen, die von externen Dateien abhängen, sind `B` -Variablen "lokal gebunden". Sie sind der erste Schritt zur **Parametrisierung** innerhalb deiner YAML-Strukturen, noch bevor wir zu den externen CSV-Daten (`T` -/XL-Variablen) übergehen.

DSL Keywords – Dynamische Datenmanipulation

Während Variablen (`{E[...]}` und `{B[...]}`) statische Werte in dein YAML-Template laden, dienen die **Keywords** dazu, diese Daten während der Laufzeit zu verarbeiten oder zu generieren.

1. String Keywords

Diese dienen zur gezielten Manipulation von Textinhalten oder zur Simulation von speziellen Eingaben.

Keyword	Beschreibung / Funktion	Beispiel	Resultat (Log)
<code>STRING:NULL</code>	Erzeugt einen leeren Wert (bzw. entfernt die Direktive).	<code>echo</code> <code>STRING:NULL</code>	<code>"</code> / (leer)
<code>STRING:FIX:N</code>	Erzeugt einen String aus 'A' mit	<code>echo</code> <code>STRING:FIX:7</code>	<code>'AAAAAAA'</code>


```

40 >>> DSL      command: 'echo BOOLEAN:false'
41 >>> DSL      response: buff_out
42 >>> CMD      : echo false
43 <<< OUT      : false
44 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 32 ms
45
46 >>> DSL      # 6
47 >>> DSL      - type: CMD
48 >>> DSL      op: EXEC
49 >>> DSL      command: 'echo STRING:EMPTY G'
50 >>> DSL      response: buff_out
51 >>> CMD      : echo G
52 <<< OUT      : G
53 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 32 ms
54
55 >>> DSL      # 7
56 >>> DSL      - type: CMD
57 >>> DSL      op: EXEC
58 >>> DSL      command: 'echo STRING:5:EMPTY G'
59 >>> DSL      response: buff_out
60 >>> CMD      : echo G
61 <<< OUT      : G
62 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 31 ms
63

```

Special Runtime Keywords: {NULL} und {EMPTY}

Während die bisher behandelten `STRING`-Keywords (`STRING:NULL` , `STRING:EMPTY`) bereits **beim Laden des Testfalls (Loadtime)** in das YAML-Template integriert und ersetzt werden, gibt es spezielle Keywords für die Schnittstellenkommunikation: `{NULL}` und `{EMPTY}` .

Der Unterschied: Wann erfolgt die Ersetzung?

- **Loadtime-Keywords (z.B. `STRING:NULL`):** Werden vom Framework verarbeitet, sobald das YAML in den Speicher geladen wird. Sie sind statische Platzhalter.
- **Runtime-Keywords (z.B. `{NULL}` , `{EMPTY}`):** Diese Keywords bleiben während des gesamten Prozesses im YAML erhalten. Sie werden erst **direkt beim Absenden** der HTTP- oder SOAP-Anfrage vom jeweiligen Schnittstellen-Adapter interpretiert.

Warum gibt es diese Unterscheidung?

Schnittstellen wie SOAP oder REST erwarten oft ganz spezifische Datenstrukturen. Wenn du ein Feld in einer XML- oder JSON-Struktur explizit als "leer" oder "null" markieren musst, darf das Framework dies nicht bereits beim Laden "wegoptimieren".

Keyword	Beschreibung / Funktion	Einsatzbereich
<code>{NULL}</code>	Signalisiert dem Adapter, den Wert als <code>null</code> zu setzen.	SOAP-Tags, JSON-Properties, Datenbank-Parameter.

<code>{EMPT}</code>	Signalisiert dem Adapter, einen leeren String <code>" "</code> zu übergeben.	Formularfelder, Pflichtfelder in XML.
---------------------	--	---------------------------------------

Beispiel in einem REST-Request:

```

1 | YAML
2 |
3 | - type: REST
4 |   op: POST
5 |   endpoint: /api/user
6 |   payload:
7 |     username: "testuser"
8 |     middleName: "{NULL}"
9 |     comment: "{EMPT}"

```

Wie funktioniert es?

Wenn der Testfall geladen wird, bleibt das YAML exakt so bestehen. Erst in der Sekunde, in der der `REST-Adapter` den Payload an den Server sendet, erkennt er:

1. `middleName` soll nicht als String `"{NULL}"` gesendet werden, sondern als JSON `null`.
2. `comment` soll als leerer String `" "` übertragen werden.

Vorteile dieser Strategie:

1. **Schnittstellen-Konformität:** Du vermeidest Probleme mit dem Schema-Validator (XSD/JSON-Schema), da die Werte exakt so übergeben werden, wie es das Protokoll erfordert.
2. **Dynamik:** Du kannst innerhalb eines Testfalls den gleichen Request-Body mehrfach verwenden, indem du `{NULL}` oder `{EMPT}` nur dort platzierst, wo du es für den spezifischen Testschritt benötigst, ohne das Template verfälschen zu müssen.
3. **Trennung der Zuständigkeiten:** Die "Business-Logik" des Testfalls bleibt sauber von den technischen Anforderungen des Transportprotokolls getrennt.

Date-Variablen

Über die einfachen `DATE`-Keywords hinaus bietet das Framework eine hochflexible Unterstützung für **erweiterte Datums-Variablen**.

Aufbau und Syntax

Die Syntax folgt dem Muster:

```

1 {DATE[base][offset][format]}.
2 {DATETIME[base][offset][format]}.

```

- **base:** (Oft leer) Basis-Datum oder Referenz.
- **offset:** Relative Verschiebung zum aktuellen Datum (z. B. `+10d` , `-1M`).
- **format:** Das gewünschte Datumsformat (Java `DateTimeFormatter`).

Die wichtigsten Anwendungsfälle

Keyword / Platzhalter	Funktion	Default-Format
<code>{DATE}</code>	Aktuelles Datum	<code>dd.MM.yyyy</code>
<code>{DATETIME}</code>	Aktuelles Datum & Zeit	<code>dd.MM.yyyy</code> <code>HH:mm:ss</code>
<code>{DATE[][]</code> <code>[ddMMyyyy]}</code>	Datum mit spezifischem Format	<code>ddMMyyyy</code>
<code>{DATE[][+10d][]}</code>	Datum + 10 Tage	<code>dd.MM.yyyy</code>
<code>{DATETIME[]</code> <code>[-200d][...]}</code>	Datum vor 200 Tagen mit Zeitstempel	<code>dd-MM-yyyy</code> <code>HH:mm:ss</code>

YAML-Beispiel zur Verifizierung

```

1 >>> DSL      # 1. Defaults ohne Parameter
2 >>> DSL      - type: CMD
3 >>> DSL      op: EXEC
4 >>> DSL      command: 'echo Default DATE: {DATE}' # Erwartet: dd.MM.y
5 >>> DSL      response: buff_out
6 >>> CMD      : echo Default 24.06.2026: 24.06.2026
7 <<< OUT      : Default 24.06.2026: 24.06.2026
8 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 81 ms
9
10 >>> DSL      - type: CMD
11 >>> DSL      op: EXEC
12 >>> DSL      command: 'echo Default DATETIME: {DATETIME}' # Erwartet:
13 >>> DSL      response: buff_out
14 >>> CMD      : echo Default 24.06.2026 00:53:16: 24.06.2026
15 <<< OUT      : Default 24.06.2026 00:53:16: 24.06.2026
16 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 31 ms
17
18 >>> DSL      # 2. Leere Klammern & Formate
19 >>> DSL      - type: CMD
20 >>> DSL      op: EXEC
21 >>> DSL      command: 'echo DATE[][][]: {DATE[][][]}' # Default Forma
22 >>> DSL      response: buff_out
23 >>> CMD      : echo 24.06.2026[][][]: 24.06.2026
24 <<< OUT      : 24.06.2026[][][]: 24.06.2026
25 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 33 ms
26
27 >>> DSL      - type: CMD
28 >>> DSL      op: EXEC
29 >>> DSL      command: 'echo DATE ddMMyyyy: {DATE[][][ddMMyyyy]}'
30 >>> DSL      response: buff_out

```



```

31 >>> CMD      : echo 24.06.2026 ddMMyyyy: 24062026
32 <<< OUT      : 24.06.2026 ddMMyyyy: 24062026
33 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 32 ms
34
35 >>> DSL      - type: CMD
36 >>> DSL      op: EXEC
37 >>> DSL      command: 'echo DATE yyyy: {DATE[][][yyyy]}'
38 >>> DSL      response: buff_out
39 >>> CMD      : echo 24.06.2026 yyyy: 2026
40 <<< OUT      : 24.06.2026 yyyy: 2026
41 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 30 ms
42
43 >>> DSL      # 3. Offsets mit Default-Format
44 >>> DSL      - type: CMD
45 >>> DSL      op: EXEC
46 >>> DSL      command: 'echo Offset +10 Tage: {DATE[][+10d][]}'
47 >>> DSL      response: buff_out
48 >>> CMD      : echo Offset +10 Tage: 04.07.2026
49 <<< OUT      : Offset +10 Tage: 04.07.2026
50 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 31 ms
51
52 >>> DSL      - type: CMD
53 >>> DSL      op: EXEC
54 >>> DSL      command: 'echo Offset +6M +1d: {DATE[][+6M+1d][]}'
55 >>> DSL      response: buff_out
56 >>> CMD      : echo Offset +6M +1d: 25.12.2026
57 <<< OUT      : Offset +6M +1d: 25.12.2026
58 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 32 ms
59
60 >>> DSL      # 4. Offsets mit non-Default-Format
61 >>> DSL      - type: CMD
62 >>> DSL      op: EXEC
63 >>> DSL      command: 'echo Offset +10 Tage: {DATE[][+10d][]}'
64 >>> DSL      response: buff_out
65 >>> CMD      : echo Offset +10 Tage: 04.07.2026
66 <<< OUT      : Offset +10 Tage: 04.07.2026
67 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 26 ms
68
69 >>> DSL      - type: CMD
70 >>> DSL      op: EXEC
71 >>> DSL      command: 'echo Offset +2d for 22.01.2025: {DATE[22.01.20
72 >>> DSL      response: buff_out
73 >>> CMD      : echo Offset +2d for 22.01.2025: 24.01.2025
74 <<< OUT      : Offset +2d for 22.01.2025: 24.01.2025
75 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 33 ms
76
77 >>> DSL      # 5. Komplexe DATETIME
78 >>> DSL      - type: CMD
79 >>> DSL      op: EXEC
80 >>> DSL      command: 'echo DATETIME -200d: {DATETIME[][ -200d][dd-MM-
81 >>> DSL      response: buff_out
82 >>> CMD      : echo 24.06.2026 00:53:16 -200d: 06-12-2025 00:53:16
83 <<< OUT      : 24.06.2026 00:53:16 -200d: 06-12-2025 00:53:16
84 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 34 ms
85

```

Parametrisierung von Testfällen (Variables vs. Direct Access)

Die Flexibilität unseres Frameworks erlaubt es dir, Daten auf zwei Arten in deine Testschritte einzubinden. Beide Methoden führen zum gleichen Ergebnis, bieten aber unterschiedliche Vorteile je nach Komplexität deines Testfalls.

1. Variablen-Block (Kapselung)

Der `variables`-Block dient dazu, Werte lokal im Testfall zu definieren. Dies ist besonders mächtig, wenn du Werte aus globalen Konfigurationen (`{E[...]}`) oder dynamische Zeitstempel (`{DATE[...]}`) als lokale Variable "verpacken" möchtest.

- **Vorteil:** Du kannst komplexe oder lange Pfade/Werte zentral definieren und im Testschritt einfach über `{B[...]}` abrufen.

```
1 | YAML
  |
  | - id: TC Variables
  |   description: Testfall mit Variablen-Block
  |   variables:
  |     db_user: "{E[env.database.user]}"
  |     aktuelles_datum: "{DATE[...]}"
  |   steps:
  |     - type: CMD
  |       op: EXEC
  |       command: 'echo db_user: {B[db_user]}'
  |     - type: CMD
  |       op: EXEC
  |       command: 'echo Datum: {B[aktuelles_datum]}'
  |
```

2. Direkter Zugriff (Inline)

Du bist nicht dazu gezwungen, Variablen zu definieren. Wenn ein Wert nur einmal benötigt wird oder es sich um einen globalen Standard handelt, kannst du ihn direkt im `command` aufrufen.

- **Vorteil:** Weniger Overhead, kürzere YAML-Dateien und sofortige Übersichtlichkeit bei einfachen Tests.

```
1 | YAML
  |
  | - id: TC Variables
  |   description: Testfall mit direktem Zugriff
  |   variables: {} # Block kann leer bleiben oder ganz entfallen
  |   steps:
  |     - type: CMD
  |       op: EXEC
  |       command: 'echo Datum: {DATE[...]}
  |
  |     - type: CMD
  |       op: EXEC
  |       command: 'echo db_user: {E[env.database.user]}'
  |
```

Unterschied: Variablen vs. Dynamische Keywords

Es ist entscheidend zu verstehen, wann genau ein Wert berechnet wird. Ein häufiger Fehler ist die Annahme, dass eine Variable in `variables:` sich bei jedem Aufruf aktualisiert – das ist jedoch nicht der Fall.

Das Prinzip "Loadtime vs. Runtime"

- `{B[...]}` (Variablen) : Diese werden beim Laden des Testfalls (Initialisierung) einmalig aufgelöst. Der Wert ist für die gesamte Dauer des Testlaufs "eingefroren".
- `{DATETIME}` / `{DATE}` (Keywords) : Diese werden bei jedem einzelnen Schritt (Runtime) neu berechnet, genau in dem Moment, in dem der Befehl an die Engine gesendet wird.

Beispiel: TC Variables 3 - Timestamp-Differenz

Dieses Beispiel verdeutlicht, warum `{B[datetime]}` nach 1 Sekunde immer noch den alten Wert anzeigt, während `{DATETIME}` die aktuelle Systemzeit liefert.

1 | YAML

```
1 - id: TC Variables 3 - timestamp difference
2   description: Vergleich zwischen statischen B-Variablen und dynamisc
3   variables:
4     datetime: "{DATETIME}" # Wert wird beim Start "eingefroren"
5   steps:
6     # Step 1: Initialer Wert
7     - type: CMD
8       op: EXEC
9       command: 'echo time: {B[datetime]}' # Ausgabe: 01:00:00 (Statis
10
11    # Step 2: Dynamischer Wert
12    - type: CMD
13      op: EXEC
14      command: 'echo time: {DATETIME}' # Ausgabe: 01:00:00 (Aktuel
15
16    - type: WAIT
17      value: 1000 # Warte 1 Sekunde
18
19    # Step 4: B[datetime] ist immer noch gleich wie in Step 1
20    - type: CMD
21      op: EXEC
22      command: 'echo time: {B[datetime]}' # Ausgabe: 01:00:00 (Wert h
23
24    # Step 5: DATETIME zeigt die neue Zeit
25    - type: CMD
26      op: EXEC
27      command: 'echo time: {DATETIME}' # Ausgabe: 01:00:01 (Aktuel
28
```

Was du dir merken musst:

Statik vs. Dynamik: Wenn du einen Zeitstempel benötigst, der **über alle Schritte hinweg konsistent** bleiben soll (z. B. für eine Korrelations-ID oder einen Referenzzeitpunkt), nutze eine Variable im `variables`-Block. Wenn du die **exakte Systemzeit zum Zeitpunkt der Ausführung** benötigst, nutze immer das direkte Keyword `{DATETIME}`.

Dieses Verhalten ist kein Bug, sondern ein Feature: Es ermöglicht dir, Zeitpunkte innerhalb eines Testfalls "anzuhalten", um sie beispielsweise in mehreren Requests als identischen Zeitstempel zu verwenden.

`{R[...]}` (Reusable) Variables

Die `{R[...]}`-Syntax ist exklusiv für den Austausch von Werten in **Reusable Teststeps** (via `type: CALL`) reserviert. Dies dient der sauberen Trennung zwischen lokalen Variablen (`B`-Buffer) und expliziten Parametern, die an ein Modul übergeben werden.

- **Beispiel für die Übergabe:**

```
1 | YAML
2 |
3 | - type: CALL
4 |   command: "reusable.logcheck"
5 |   parameters:
6 |     service: "doan"
   |     code: "ZS-00"
```

- **Verwendung:** Innerhalb des Reusable-Modules kannst du dann über `{R[service]}` auf den übergebenen Wert zugreifen.

Mehr Details dazu folgen im Kapitel "Reusable Module".

`{XL[...]}` (Excel/CSV-Injection) Variables

Die `{XL[...]}`-Variablen sind ein spezialisiertes Feature für **Templates**. Sie ermöglichen den direkten Zugriff auf Spaltenwerte einer externen Datendatei (z. B. CSV) während der Instanziierung.

- **Einschränkung:** Diese dürfen **ausschließlich in Templates** verwendet werden. Sie dienen dazu, beim "Durchlaufen" einer Tabelle (`Data-Driven`) jeden Datensatz nahtlos in die Testschritte zu injizieren.

Mehr Details dazu folgen im Kapitel "Templates und Instanziierung".